Assessment with Product Framing

Expectations

This exercise is intentionally designed to assess how you think, design, and build software in an AI-driven environment.

You are required to use AI tools to accelerate your work. We care about how you use them, the clarity of your thinking, and the quality of your engineering decisions.

We expect you to:

- Demonstrate clarity in thought and structured problem solving
- Show strong engineering fundamentals & product thinking
- Make thoughtful architectural and design decisions
- Write production-quality code and tests
- Use AI intentionally while maintaining correctness and quality

Please make incremental commits so we can understand how your solution evolved.

Goal:

Build employee salary management software for an organization with 10,000 employees.

User Persona:

HR Manager of the org

Problem Statement:

Currently, ACME org's HR team manages salary data for 10,000 employees across multiple countries, with everything managed via excels, which is tedious. We want the HR manager to manage the salaries data via web-based software and be able to answer questions about how the org pays people.

Requirements:

Write a one-page requirements document before building the software, outlining the goal, scope & features, and what you are deliberately leaving out, and reasoning for it.

Technical Constraints:

You should build end-to-end, fully functional software, including backend & UI.

1. Backend:

- {Language & Framework as per the role you've applied for (preferred) or any other framework of your choice. Relational database of your choice, like SQLite.

2. UI:
- ReactJS or NextJS. AngularJS for Java
- Component library of your choice

3. Seeding:
- Seed script with 10,000 employees.

4. Readiness:
- Fully functional deployed software
- Any video demo of the software

Development Approach:

Using agentic AI tools of your choice, build the solution while maintaining high standards of code quality and test coverage.

Your solution should include:

- A meaningful set of unit tests that cover the core functionality
- Tests that are fast, deterministic, and easy to understand
- Good code structure, readability, and maintainability

Your commit history should show the evolution of your solution through incremental commits. This helps us understand how you approached the problem and how your implementation developed over time.

Artifacts

Along with your solution, please commit any artifacts that help us understand your thinking and approach.
Examples might include:

- Requirements document
- Planning or design notes
- Architecture diagrams
- Prompts or instructions used with AI tools
- Trade-off explanations
- Performance considerations

There is no required format — use whatever helps best explain your approach.

How to Succeed in This Assessment

The strongest submissions typically show:

- Clear thinking and structured problem solving
- Thoughtful architecture and design decisions
- Clean, maintainable code
- Meaningful tests
- Intentional use of AI tools

- Evidence of product thinking

We are not looking for the most complex system, but for good engineering judgment.

Submission Instructions

- Commit your code to a Git repository
- Ensure commits show your development process
- Include all artifacts mentioned above
- Share the repository link by replying to the same email once you are done